

Decision Logic Roadmap

Project Drisya Decision Logic

Suwarna Pyakurel

Decision Logic Roadmap For Project Drisya

Owner: Suwarna Pyakurel

Module: `modules/decision_logic`

Project: Drisya, an autonomous search-and-rescue robot using simulation, WiFi CSI human detection, navigation, mapping, and a shared Blackboard.

1. Mission Of This Module

The Decision Logic module is the robot's high-level brain.

It reads system state from the shared Blackboard, decides the next mission intent, and writes commands or targets for other modules to execute.

It should not directly build the WiFi detector, occupancy grid mapper, simulator, or motor firmware. It should coordinate them through clean Blackboard keys.

2. What You Must Build

Your final module should deliver:

- A 10 Hz runtime loop in `brain.py`.
- A safety-first Behavior Tree using `py_trees`.
- Mission states: idle, searching, navigating, confirming victim, returning, stopped.
- Victim response logic using WiFi detection confidence.
- Exploration behavior when no victim is detected.
- Explainability traces showing why each action was selected.
- Optional PPO reinforcement learning experiments for exploration, not for emergency safety.
- Unit and scenario tests proving the decision rules work.

3. What You Should Not Build

Do not take ownership of these parts:

- WiFi CSI CNN model.
- Triangulation math owned by WiFi detection.
- Occupancy grid mapping.
- A* pathfinding.
- Pygame dashboard rendering.
- ESP32 low-level motor PWM code.

Your module can request navigation targets and write motor commands only through agreed shared keys.

4. Current Repository Reality

Based on the repo:

- `modules/decision_logic/README.md` exists.
- Implementation files such as `brain.py`, `behavior_tree/`, `rl_env/`, and `train_ppo.py` do not exist yet.
- `shared/blackboard.py`, `shared/mock_detection.py`, `shared/mock_navigation.py`, and `shared/sensor_form` currently describe intent but do not yet provide full working APIs.
- `modules/navigation/README.md` and `modules/wifi_detection/README.md` are currently empty.
- `modules/simulation_viz/README.md` defines how the simulator expects to read decision state and motor commands.

This means your first real engineering job is to make the Decision Logic contract precise and testable.

5. Recommended Final Structure

Build this structure gradually:

```
modules/decision_logic/  
+-- README.md  
+-- brain.py  
+-- config.py  
+-- contracts.py  
+-- state_snapshot.py  
+-- explainability.py  
+-- behavior_tree/  
|   +-- __init__.py  
|   +-- tree_factory.py
```

```

|   +-- conditions.py
|   +-- actions.py
|   +-- policies.py
+-- rl_env/
|   +-- __init__.py
|   +-- sar_explore_env.py
|   +-- rewards.py
|   +-- wrappers.py
+-- experiments/
|   +-- scenario_runner.py
|   +-- metrics.py
+-- tests/
|   +-- test_contracts.py
|   +-- test_behavior_tree_safety.py
|   +-- test_detection_response.py
|   +-- test_exploration_policy.py
|   +-- test_explainability.py
+-- train_ppo.py

```

6. Architecture

Use a layered architecture:

```

Blackboard State
|
v
State Snapshot Validation
|
v
Safety-First Behavior Tree
|
+--> Emergency stop
+--> Obstacle/stale-data safety
+--> Victim confirmation
+--> Navigation target
+--> Exploration policy
+--> Idle
|
v
Blackboard Outputs + Explainability Trace

```

The Behavior Tree should always supervise any RL policy. RL may suggest exploration actions, but it should never override emergency stop, obstacle stop, or stale-data safety.

7. Behavior Tree Design

Recommended top-level priority order:

```

Root: Priority Selector
+-- EmergencyStopSequence
|   +-- IsEmergencyStopRequested?
|   +-- WriteStopCommand
+-- SafetyAvoidanceSequence
|   +-- IsDataStale? OR IsObstacleTooClose?
|   +-- WriteSafeCommand
+-- VictimConfirmationSequence
|   +-- IsHighConfidenceDetection?
|   +-- SetVictimTarget

```

```
|  +-- RequestConfirmation
+-- NavigateToTargetSequence
|  +-- HasActiveTarget?
|  +-- IsPathAvailable?
|  +-- PublishNavigationIntent
+-- ExploreSequence
|  +-- IsMissionRunning?
|  +-- SelectFrontierOrPolicyAction
|  +-- PublishExplorationIntent
+-- IdleAction
```

Required condition nodes:

- IsEmergencyStopRequested
- IsMissionRunning
- IsDataStale
- IsObstacleTooClose
- IsHighConfidenceDetection
- HasActiveTarget
- IsPathAvailable
- IsRobotStuck

Required action nodes:

- WriteStopCommand
- WriteSafeCommand
- SetVictimTarget
- RequestConfirmation
- PublishNavigationIntent
- PublishExplorationIntent
- WriteIdleState
- WriteDecisionTrace

8. Initial Thresholds

Put these in `config.py` later. They are starting values, not final truth.

Parameter	Suggested value	Why
TICK_HZ	10	Matches decision module README.
DETECTION_CONFIDENCE_HIGH	0.75	Avoid chasing weak detections.
DETECTION_CONFIDENCE_CONFIRM	0.60	Useful for confirmation state.
OBSTACLE_STOP_CM	20	Conservative small-robot stop distance.
OBSTACLE_SLOW_CM	35	Early avoidance zone.
DATA_STALE_MS	1000	Avoid acting on old sensor state.
DEFAULT_FORWARD_SPEED	40	Moderate initial speed.
DEFAULT_TURN_SPEED	30	Conservative turn speed.

9. Implementation Timeline

Week	Focus	Output
1	Understand repo and freeze Blackboard contract	Draft contract accepted by team
2	Build typed input validation	<code>contracts.py</code> , <code>state_snapshot.py</code> , tests
3	Build safety Behavior Tree	Emergency stop and obstacle safety
4	Add victim detection response	Confidence-gated target logic
5	Add deterministic exploration	Frontier/fallback baseline
6	Build 10 Hz runtime loop	<code>brain.py</code> dry-run mode
7	Add explainability traces	JSONL/Blackboard trace outputs
8	Scenario tests with mocks	9 repeatable test scenarios
9-10	Build Gymnasium environment	<code>SARExploreEnv</code>
11-12	Train PPO and compare to baseline	Reproducible metrics
13	Integrate with simulator	Dashboard can show behavior state
14	Harden motor command safety	Clamp, duration limit, watchdog
15	Run scenario evaluation	Metrics table
16	Documentation and defense prep	Clean README, diagrams, limitations

10. Testing Strategy

Minimum unit tests:

- Contract validation.
- Emergency stop priority.
- Obstacle safety priority.
- Stale-data safe stop.
- Detection confidence thresholds.
- Exploration target selection.
- Motor command clamping.
- Explainability trace formatting.

Minimum scenario tests:

- Mission idle.
- Mission running with no victim.
- Obstacle in front.
- Emergency stop requested.
- Low-confidence detection.
- Medium-confidence detection.
- High-confidence detection.
- Path unavailable.
- Stale sensor data.

Metrics to collect:

- Decision tick latency.
- Exploration coverage.
- Collision count.
- Unsafe proximity count.
- Victim target response time.
- Rescue success rate in simulation only.
- Emergency override count.

11. Definition Of Done

Your Decision Logic portion is complete when:

- It runs independently with mock data.
- It runs with the simulator through Blackboard keys.
- It writes motor commands in the agreed format.
- It prioritizes emergency stop and obstacle safety.
- It reacts correctly to victim detection confidence.
- It explores when no victim is detected.
- It logs explainable decision traces.
- It has unit tests for core behavior nodes.
- It has scenario tests for mission-level behavior.
- It has measured simulation metrics.
- It documents limitations honestly.

12. Defense Explanation

The Decision Logic module uses a layered architecture. A deterministic Behavior Tree runs at 10 Hz and always evaluates safety before mission behavior. If emergency stop, stale sensor data, or unsafe obstacle proximity is detected, it immediately writes a safe stop or avoidance command. If the system is safe and WiFi detection reports a high-confidence human location, it publishes a victim target and requests confirmation. If no victim is detected, the robot explores unknown regions using a deterministic baseline first and an optional RL policy later. Every selected action is logged with a reason, making the robot's behavior explainable for the dashboard and final defense.